Research Project in Application of Artificial Intelligence on Medical Imaging
Lab 2: EEG Classification with EEGNet

Chia-Hsuan Hsu Student ID: 315511079

Fall 2026

1 GitHub Link

https://github.com/tongyu0924/NYCU-MedAI-115-1/tree/main/lab2

2 Introduction

Electroencephalography (EEG) records the electrical activity of the brain through electrodes on the scalp. In a motor-imagery brain–computer interface (BCI), a subject imagines moving the left or the right hand, and the system must decide which hand was imagined from a short window of noisy, low-amplitude signals. This lab uses dataset IIIb of BCI Competition III [4] (subjects S4 and X11). Each trial contains 2 bipolar EEG channels and 750 time samples. There are two classes: left hand and right hand. The training and testing sets each contain 1,080 trials (540 per subject), and both classes are exactly balanced. Missing samples (NaN) are replaced by the mean signal value, and every trial is reshaped to a $1 \times 2 \times 750$ input.

EEGNet [1] is a compact convolutional neural network designed for EEG. Its structure follows the classical filter-bank common-spatial-pattern pipeline, but it learns every stage end-to-end:

1. a **temporal convolution** whose kernels act as learned band-pass filters (for example, filters that pass the μ and β rhythms that change during motor imagery);
2. a **depthwise convolution** across the electrodes that learns a separate spatial filter for each temporal filter;
3. a **separable convolution** that summarises each feature map over time and then mixes the maps; and
4. a single linear classification layer.

Batch normalisation, ELU activations, average pooling and dropout are inserted between the blocks. The resulting network has only 17,874 trainable parameters, about 8× fewer than DeepConvNet (150,977). This small size makes overfitting less likely when only 1,080 trials are available for training.

In this lab I implemented EEGNet from scratch in PyTorch, as well as DeepConvNet [2] for the bonus. I then ran 34 training runs to study how the following factors affect testing accuracy:

- the activation function and the ELU α;
- the dropout rate;
- the learning rate and its schedule;
- a true depthwise-separable convolution;
- data augmentation; and
- the random seed.

The best model reached a **highest testing accuracy of 88.06%**.

3 Experiment Setups and Results

3.1 Model and default hyper-parameters

Table 1 lists the EEGNet architecture as implemented, which is identical to the reference model in the lab specification. The default hyper-parameters follow the specification:

- batch size 64, Adam with learning rate 10^{-2} and weight decay 10^{-3}, and 150 epochs;
- loss: `nn.CrossEntropyLoss()`, with no softmax after the last layer;
- ELU with $\alpha = 1$ and dropout $p = 0.25$.

After every epoch, the model is evaluated on the full testing set. The weights from the epoch with the highest testing accuracy are saved with `copy.deepcopy(model.state_dict())`. The template code stored `model.state_dict()` directly; that is a live reference, so it would actually have saved the last epoch. All runs used seed 0 unless stated otherwise and ran on a Colab GPU.

Table 1: EEGNet as implemented (input $[B, 1, 2, 750]$, 17,874 parameters).

Block	Layers	Output shape
`firstconv`	Conv2d($1 \to 16$, $(1, 51)$, pad $(0, 25)$, no bias) + BN	$16 \times 2 \times 750$
`depthwiseConv`	Conv2d($16 \to 32$, $(2, 1)$, groups $= 16$) + BN + ELU + AvgPool$(1, 4)$ + Dropout	$32 \times 1 \times 187$
`separableConv`	Conv2d($32 \to 32$, $(1, 15)$, pad $(0, 7)$) + BN + ELU + AvgPool$(1, 8)$ + Dropout	$32 \times 1 \times 23$
`classify`	Flatten + Linear($736 \to 2$)	2 logits

3.2 Round 1: settings from the specification (150 epochs)

The first round changed one factor at a time with respect to the baseline. Table 2 reports three numbers for each run:

- the *highest* testing accuracy over all epochs, which is the metric used for grading;
- the *mean testing accuracy of the last 10 epochs*, which is a less optimistic estimate because a single peak of a noisy curve is partly luck; and
- the final training accuracy, which indicates under- or overfitting.

Table 2: Round 1 results for EEGNet (150 epochs, seed 0). The baseline row uses ELU with $\alpha = 1.0$, dropout 0.25 and Adam with a constant learning rate of 10^{-2}.

Factor	Setting	Highest test (%)	Epoch	Mean last 10 (%)	Final train (%)
Baseline	ELU, $\alpha = 1.0$	81.57	71	79.04	88.89
Activation	ReLU	**86.11**	92	**83.81**	95.09
	LeakyReLU (0.01)	83.70	128	81.85	94.81
ELU α	$\alpha = 0.1$	**86.48**	136	**83.94**	95.56
	$\alpha = 0.5$	83.52	139	81.16	93.89
	$\alpha = 1.5$	76.67	148	75.24	82.41
Dropout	$p = 0.1$	81.20	111	78.10	96.76
	$p = 0.5$	75.00	110	72.20	75.09
Learning rate	lr 10^{-3}	81.85	31	80.13	95.93
	lr 10^{-2} + cosine annealing	82.31	76	81.24	94.26
Architecture	true depthwise-separable conv	79.91	113	78.89	90.93

Figures 1–4 compare the curves of each group. To make the trends readable, every curve is smoothed with a 5-epoch moving average. The unsmoothed curves of each run are produced by `main.py` and are available in the GitHub repository.

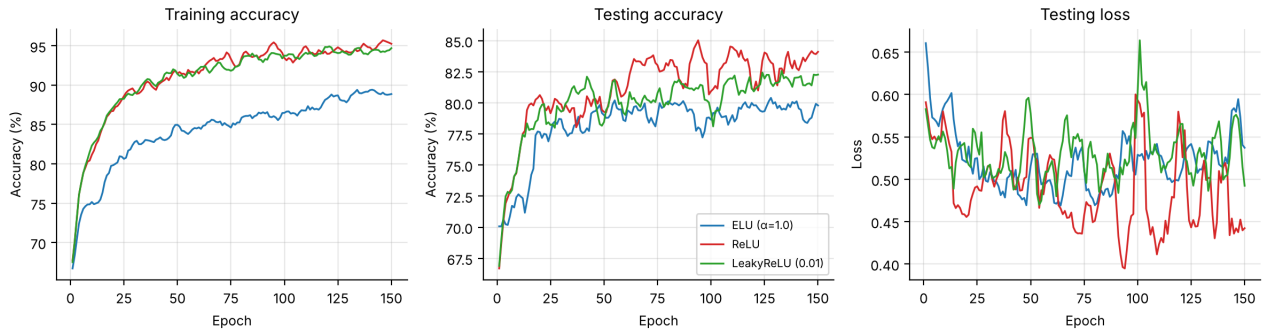


Figure 1: Activation functions: ELU ($\alpha = 1$), ReLU and LeakyReLU.

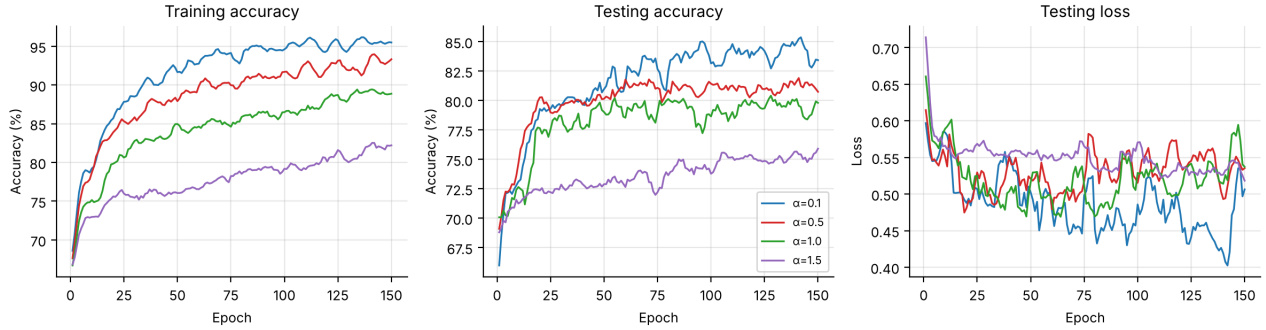


Figure 2: ELU with $\alpha \in \{0.1, 0.5, 1.0, 1.5\}$. Smaller α gives monotonically higher training and testing accuracy.

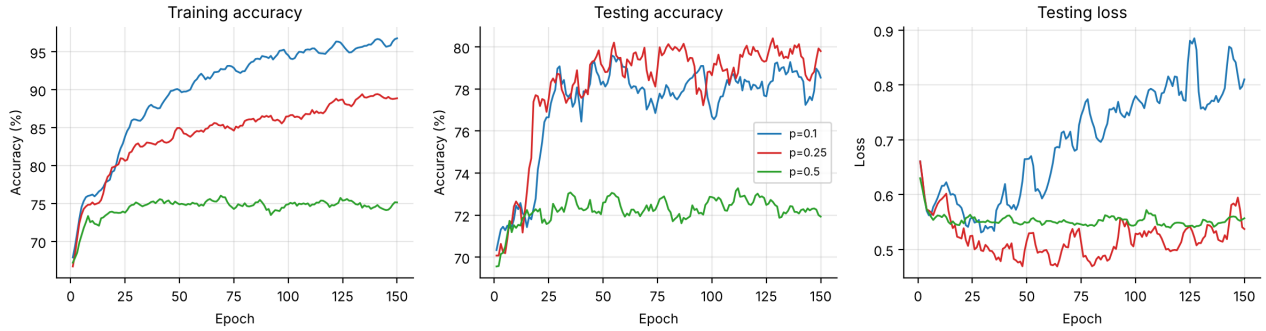


Figure 3: Dropout rate. With $p = 0.5$ the model underfits; with $p = 0.1$ the testing loss rises again after about 35 epochs, which indicates overfitting.

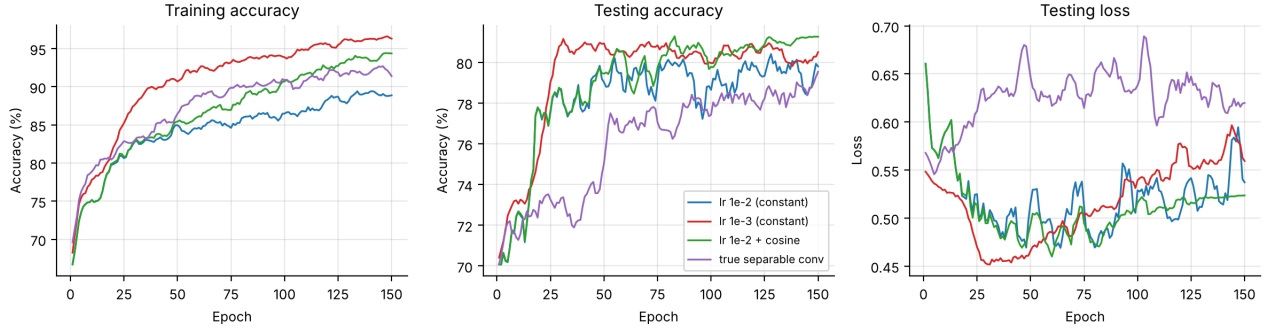


Figure 4: Learning rate and schedule, and the true depthwise-separable convolution (lr 10^{-2}, constant).

3.3 Round 2: improving the best settings (300 epochs)

Round 1 showed that the activation function matters most: ReLU and ELU with a small α were clearly the best. The baseline was also still improving at epoch 150. Round 2 therefore started from ELU with $\alpha = 0.1$ or ReLU, trained for 300 epochs, and added the following techniques (all implemented in `main.py` as command-line options):

- **Cosine annealing**: the learning rate decays from 10^{-2} to 0 over training.
- **Data augmentation**, applied to the training batches only:
  - a random circular time shift of up to ± 25 samples (0.1 s);
  - random amplitude scaling by up to $\pm 10\%$; and
  - Gaussian noise with a standard deviation of 5% of the signal standard deviation.
- **Other options**: label smoothing, AdamW, and a wider EEGNet ($F_1 = 32$, $F_2 = 64$, 66,466 parameters).

The script `tune.py` first runs ten candidate settings (Table 3). It then retrains the three best settings with seeds 1–3 to check that their accuracy is not due to a lucky initialisation (Figure 6).

Table 3: Round 2 results for EEGNet (300 epochs, seed 0). Unless stated otherwise, lr is 10^{-2} and weight decay is 10^{-3}. "Aug." denotes the time shift, amplitude scaling and noise augmentation.

Setting	Highest test (%)	Epoch	Mean last 10 (%)	Final train (%)
ELU α=0.1 + cosine + aug.	**88.06**	234	**86.99**	97.04
ELU α=0.1, lr 10^{-3}, wd 10^{-4}	87.78	270	86.79	99.07
ReLU, lr 10^{-3}, wd 10^{-4}	87.69	224	86.28	99.07
ReLU + cosine + aug.	87.59	179	86.77	97.50
ELU α=0.1 + cosine + aug. + label smoothing 0.1	86.85	273	85.64	96.39
ReLU + cosine	86.85	224	86.57	99.91
ELU α=0.1 + cosine	86.76	222	86.42	99.63
ReLU + AdamW (wd 0.01) + cosine + aug.	86.76	273	86.49	98.24
ReLU, wide EEGNet (F_1=32, F_2=64) + cosine + aug.	86.67	264	85.56	98.06
LeakyReLU + cosine	85.00	277	84.47	99.35

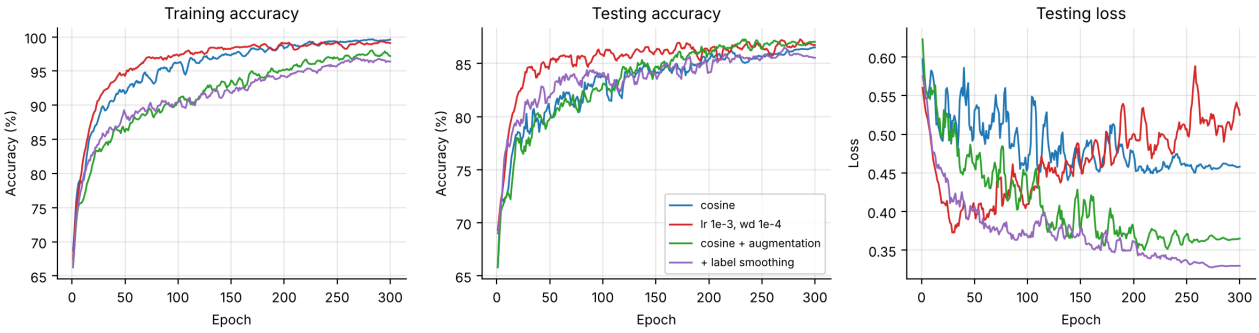


Figure 5: Round 2 runs with ELU ($\alpha = 0.1$) and 300 epochs. Augmentation keeps the testing loss lowest and the testing accuracy highest at the end of training.

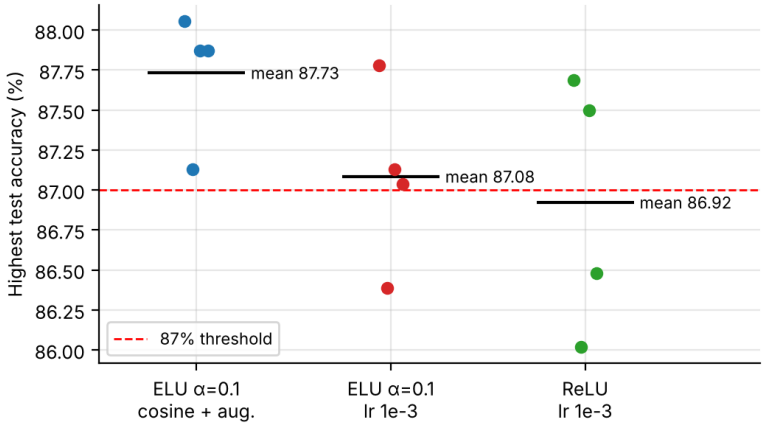


Figure 6: Highest testing accuracy of the three best round-2 settings over four seeds (0–3); the black bars are the means. All four seeds of the best setting are above 87%.

3.4 Highest testing accuracy

The highest testing accuracy is **88.06%**. It was reached at epoch 234 by EEGNet with ELU ($\alpha = 0.1$), dropout 0.25, Adam (lr 10^{-2}, weight decay 10^{-3}) with cosine annealing over 300 epochs, and data augmentation. The command was:

```
python main.py -activation elu -alpha 0.1 -num_epochs 300 -cosine \
               -aug_shift 25 -aug_scale 0.1 -aug_noise 0.05 -seed 0
```

Figure 7 shows the saved best checkpoint reloaded in Colab and evaluated on the testing set; it reproduces 88.06%.

```
/content/lab2
(1080, 1, 2, 750) (1080,) (1080, 1, 2, 750) (1080,)
EEGNet(
  (firstconv): Sequential(
    (0): Conv2d(1, 16, kernel_size=(1, 51), stride=(1, 1), padding=(0, 25), bias=False)
    (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
  (depthwiseConv): Sequential(
    (0): Conv2d(16, 32, kernel_size=(2, 1), stride=(1, 1), groups=16, bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=0.1)
    (3): AvgPool2d(kernel_size=(1, 4), stride=(1, 4), padding=0)
    (4): Dropout(p=0.25, inplace=False)
  )
  (separableConv): Sequential(
    (0): Conv2d(32, 32, kernel_size=(1, 15), stride=(1, 1), padding=(0, 7), bias=False)
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ELU(alpha=0.1)
    (3): AvgPool2d(kernel_size=(1, 8), stride=(1, 8), padding=0)
    (4): Dropout(p=0.25, inplace=False)
  )
  (classify): Sequential(
    (0): Linear(in_features=736, out_features=2, bias=True)
  )
)
Best weights: weights/tune_a0.1_aug300_s0.pt
Highest testing accuracy: 88.06%  (test loss 0.3453)
```

Figure 7: Screenshot: the best checkpoint (`weights/tune_a0.1_aug300_s0.pt`) reloaded and evaluated on the 1,080 testing trials.

3.5 Training and testing loss and accuracy curves

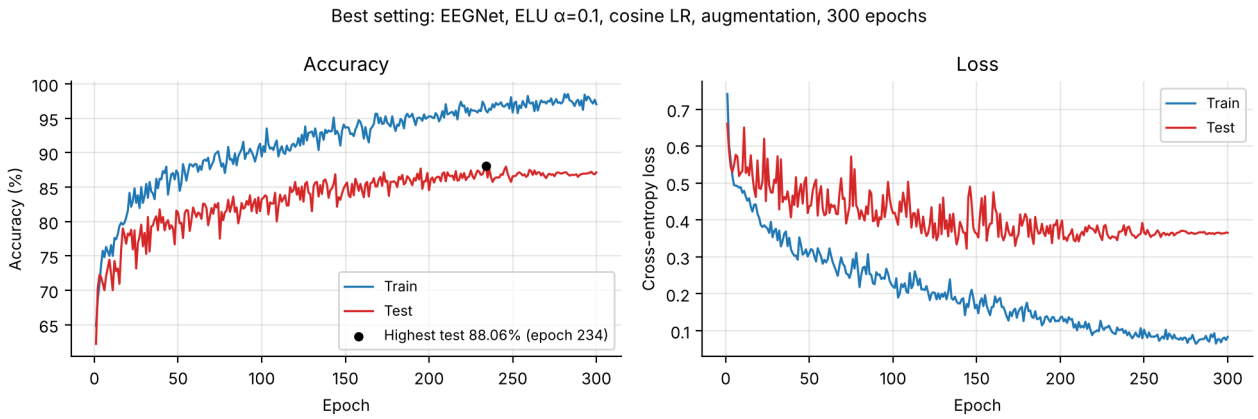


Figure 8: Training and testing accuracy and loss of the best model. The black dot marks the highest testing accuracy.

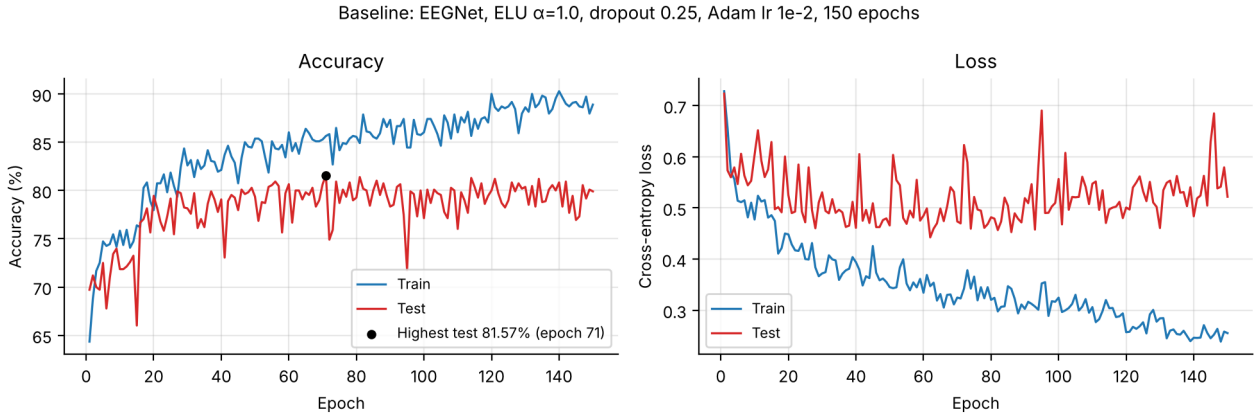


Figure 9: Training and testing accuracy and loss of the baseline EEGNet from the specification.

4 Discussion

4.1 Findings

The activation function and ELU α matter most. In round 1, accuracy increased steadily as α decreased. Comparing $\alpha = 1.5$ with $\alpha = 0.1$:

- the highest test accuracy rose from 76.67% to 86.48%;
- the mean test accuracy of the last 10 epochs rose from 75.24% to 83.94%; and
- the final training accuracy rose from 82.41% to 95.56%.

ReLU, which corresponds to the limit $\alpha \to 0$, behaved almost the same as $\alpha = 0.1$ (86.11%). The training curves (Figure 2) show that a large α mainly slows down *fitting*: it is an optimisation effect, not overfitting. Every activation follows a BatchNorm layer, so roughly half of its inputs are negative. With a large α, these inputs produce outputs down to $-\alpha$ instead of approximately 0. The following average-pooling layer therefore mixes large negative and positive values, which blurs the features passed to the next block. In addition, under weight decay 10^{-3} with lr 10^{-2}, the network fits the data less easily. A small α keeps the useful property of ELU (a non-zero gradient for $x < 0$, so units never "die") while producing sparser, ReLU-like features. LeakyReLU (slope 0.01) was slightly worse than ReLU in both rounds.

Dropout controls the bias–variance trade-off. With $p = 0.5$ the model underfits: it reaches only 75.09% training accuracy and 75.00% at best on the test set. With $p = 0.1$ it fits the training set better (96.76%), but the testing loss climbs from its minimum of 0.51 at epoch 34 to about 0.83 at the end, which is a clear sign of overfitting. The default $p = 0.25$ lies between these two extremes.

Learning rate and schedule. With lr 10^{-3}, the model fitted fastest. It peaked at epoch 31 and then slowly overfitted. Cosine annealing reduces the update noise at the end of training. It raised the mean of the last 10 epochs from 79.04% to 81.24% and flattened the testing curve, which makes the final model more reliable. With 300 epochs, both lr 10^{-3} (with a smaller weight decay) and lr 10^{-2} with cosine annealing exceeded 86.7%.

Data augmentation reduces overfitting. Without augmentation, the 300-epoch runs reached 99.6–99.9% training accuracy, and their testing loss rose again towards the end (Figure 5). With time shift, scaling and noise, training accuracy stayed at about 97%, and the final testing loss decreased from 0.459 to 0.365. Both the highest accuracy (88.06% vs. 86.76%) and the mean of the last 10 epochs (86.99% vs. 86.42%) improved. A random time shift makes sense for motor imagery, because the onset of the imagined movement is not perfectly aligned across trials.

Things that did not help.

- **A wider EEGNet** (3.7× the parameters) did not improve accuracy (86.67%). The 1,080 training trials, not the model capacity, are the limiting factor.
- **Label smoothing** lowered the peak to 86.85%.
- **AdamW** with a stronger weight decay gave no clear gain.

The result is robust to the seed, but the "highest" metric is optimistic. Over seeds 0–3, the best setting reached 88.06, 87.13, 87.87 and 87.87% (mean 87.73%), so every seed passed 87%. The other two candidates averaged 87.08% and 86.92%. Training is also not bit-exact on a GPU. Repeating the baseline gave 81.57% and 81.94%, and a re-run of the best setting gave 86.67%. The highest testing accuracy is selected *using the test set*, as this lab requires, so it is an optimistic estimate. The mean of the last 10 epochs (86.99% for the best setting) is a fairer estimate of performance on unseen data; a separate validation split would be needed for unbiased model selection.

4.2 How the model is implemented

Data. `dataloader.read_bci_data()` performs the following steps:

- concatenates the two subjects;
- maps the labels $\{1, 2\} \to \{0, 1\}$;
- reshapes each trial from $[750, 2]$ to $[1, 2, 750]$, i.e. one "image" with 2 rows (electrodes) and 750 columns (time); and
- replaces NaN values by the mean.

The path of the data folder is now resolved relative to the source file, so the code can be run from any directory. A small `Dataset` class returns float32 signals and int64 labels to a `DataLoader`; the training set is shuffled.

Model. `models/EEGNet.py` builds the four blocks in Table 1 with `nn.Sequential`:

- The temporal convolution uses a $(1, 51)$ kernel with padding 25, so the time length stays at 750.
- The depthwise convolution uses a $(2, 1)$ kernel that spans both electrodes, with `groups=16`. Each of the 16 temporal maps therefore gets $D = 2$ spatial filters (16→32 maps), and the electrode axis collapses to 1.
- The two average-pooling layers reduce the time axis $750 \to 187 \to 23$, which gives $32 \times 23 = 736$ features for `nn.Linear(736, 2)`.
- The convolutions have no bias, because each is followed by a BatchNorm layer, which provides its own shift.
- The activation is created by `get_activation(name, alpha)`, so the activation function, α, dropout rate, F_1, D, F_2 and the true-separable option are all command-line arguments.

The forward pass returns raw logits, because `CrossEntropyLoss` applies log-softmax internally.

```
self.firstconv = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=(1, 51), padding=(0, 25), bias=False),
    nn.BatchNorm2d(16))
self.depthwiseConv = nn.Sequential(
    nn.Conv2d(16, 32, kernel_size=(2, 1), groups=16, bias=False),
    nn.BatchNorm2d(32), get_activation(activation, alpha),
    nn.AvgPool2d(kernel_size=(1, 4)), nn.Dropout(p=dropout))
self.separableConv = nn.Sequential(
    nn.Conv2d(32, 32, kernel_size=(1, 15), padding=(0, 7), bias=False),
    nn.BatchNorm2d(32), get_activation(activation, alpha),
    nn.AvgPool2d(kernel_size=(1, 8)), nn.Dropout(p=dropout))
self.classify = nn.Sequential(nn.Linear(736, 2))

def forward(self, x): # [B, 1, 2, 750]
    x = self.firstconv(x) # [B, 16, 2, 750]
    x = self.depthwiseConv(x) # [B, 32, 1, 187]
    x = self.separableConv(x) # [B, 32, 1, 23]
    return self.classify(x.flatten(1)) # [B, 2] logits
```

Training and evaluation.

- **Training step**: each epoch runs `model.train()` and, for every mini-batch, augments the batch (if enabled) and then performs zero_grad → forward → loss → backward → optimizer step. The loss multiplied by the batch size and the number of correct predictions are accumulated.
- **Evaluation**: the testing set is evaluated under `model.eval()` and `torch.no_grad()`, so dropout is disabled and BatchNorm uses its running statistics.
- **Outputs**: the training and testing loss and accuracy of each epoch are saved to JSON and plotted with matplotlib. `run_experiments.py` and `tune.py` reproduce every run in this report.

4.3 The ELU function

The exponential linear unit [3] is

$$\mathrm{ELU}(x) = \begin{cases} x, & x > 0, \\ \alpha\,(e^x - 1), & x \le 0, \end{cases} \qquad \frac{d\,\mathrm{ELU}}{dx} = \begin{cases} 1, & x > 0, \\ \alpha e^x = \mathrm{ELU}(x) + \alpha, & x \le 0. \end{cases} \tag{1}$$

Compared with ReLU, ELU has three advantages:

1. Negative inputs still produce a non-zero output and gradient, so units cannot "die".

2. The negative outputs push the mean activation towards zero. This reduces the bias shift between layers and speeds up learning.

3. Unlike LeakyReLU, the negative branch saturates smoothly to $-\alpha$. Large negative inputs, such as noise, therefore give a bounded and almost constant output, which is a noise-robust "off" state.

When $\alpha = 1$, the function is also differentiable at 0. The parameter α sets the saturation level (Figure 10). As $\alpha \to 0$, ELU approaches ReLU, and as α grows, the negative region becomes deeper. In our experiments a small α (0.1) was clearly best. The price of ELU is one exponential per element, which makes it slightly slower than ReLU.

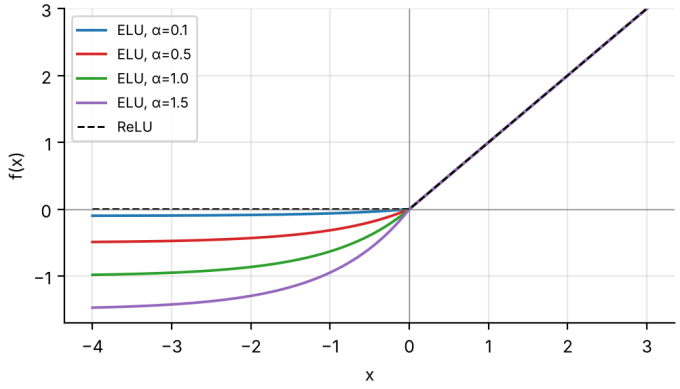


Figure 10: ELU with different α compared with ReLU.

4.4 Depthwise and separable convolution

Standard convolution. A $k_h \times k_w$ convolution from C_{in} to C_{out} channels has $C_{in}C_{out}k_hk_w$ weights. Every output channel sees every input channel, so a single layer performs both spatial (or temporal) filtering and channel mixing.

Depthwise convolution. Each input channel is convolved only with its own filter(s); in PyTorch this is `groups` $= C_{in}$. With a depth multiplier D, each input channel produces D output channels, so the layer has only $C_{in}Dk_hk_w$ weights and no information is exchanged between channels. In EEGNet the depthwise convolution has a $(2, 1)$ kernel, i.e. (number of electrodes, 1). For each of the 16 band-pass-filtered maps it learns $D = 2$ *spatial filters*, which are weighted combinations of the electrodes; this is exactly the role of common spatial patterns (CSP) in classical BCI. Because each spatial filter is tied to one frequency band, the model stays interpretable. It also uses only 64 weights instead of the 1,024 that a full convolution would need.

Depthwise-separable convolution. A standard convolution is factorised into two steps:

- a depthwise convolution, which filters each channel along time; followed by
- a pointwise 1×1 convolution, which mixes the channels.

The number of weights drops from $C_{in}C_{out}k$ to $C_{in}k + C_{in}C_{out}$. In the original EEGNet, the third block is such a separable convolution: a depthwise convolution summarises each feature map over about 0.5 s, and the 1×1 convolution learns how to combine the maps. The benefits are fewer parameters and multiply–adds, plus a built-in regularisation that suits small EEG datasets. The cost is lower expressive power, because the temporal filter of each map cannot depend on the other maps.

The model in this lab's specification uses a single full convolution, `Conv2d(32,32,(1,15))`, in the `separableConv` block. I therefore also implemented a true separable version (`-true_separable`: a depthwise $(1, 15)$ convolution with `groups=32`, followed by a 1×1 convolution). It has $10\times$ fewer

weights in that layer (Table 4) and 4,018 parameters in total instead of 17,874. Under the default settings it reached 79.91% versus 81.57%. It reached a slightly higher final training accuracy (90.93% vs. 88.89%) but a higher testing loss (minimum 0.535 vs. 0.442), so it generalised slightly worse. On this 2-channel dataset the full convolution's extra mixing capacity helps a little. Still, the separable version stays within about 2% while using 23% of the parameters, which shows how efficient the factorisation is.

Table 4: Weights of the convolution choices discussed (bias-free).

Layer	Convolution type	Weights
Spatial conv $16 \to 32$, $(2, 1)$	full convolution	1,024
Spatial conv $16 \to 32$, $(2, 1)$	depthwise, groups = 16 (EEGNet)	64
Temporal conv $32 \to 32$, $(1, 15)$	full convolution (lab specification)	15,360
Temporal conv $32 \to 32$, $(1, 15)$	depthwise (480) + pointwise (1,024)	1,504

5 Bonus: DeepConvNet

DeepConvNet [2] is a deeper, general-purpose CNN. Its first block is a temporal convolution followed by a spatial convolution. Three more blocks follow, each consisting of convolution, BatchNorm, activation, max pooling and dropout, with 50, 100 and 200 filters. A dense layer produces the output. I implemented it from the table in the specification ($C = 2$, $T = 750$, $N = 2$) without the max-norm constraint (Table 5). It has 150,977 parameters and was trained with the same hyper-parameters as the EEGNet baseline (Adam lr 10^{-2}, weight decay 10^{-3}, batch size 64, 150 epochs), with dropout 0.5.

Table 5: DeepConvNet as implemented.

Layer	Setting	Output shape
Conv2d	$1 \to 25$, $(1, 5)$	$25 \times 2 \times 746$
Conv2d + BN + act.	$25 \to 25$, $(2, 1)$	$25 \times 1 \times 746$
MaxPool + Dropout	$(1, 2)$, $p = 0.5$	$25 \times 1 \times 373$
Conv2d + BN + act. + MaxPool + Dropout	$25 \to 50$, $(1, 5)$	$50 \times 1 \times 184$
Conv2d + BN + act. + MaxPool + Dropout	$50 \to 100$, $(1, 5)$	$100 \times 1 \times 90$
Conv2d + BN + act. + MaxPool + Dropout	$100 \to 200$, $(1, 5)$	$200 \times 1 \times 43$
Flatten + Linear	$8600 \to 2$	2

Table 6: DeepConvNet compared with EEGNet (150 epochs, same optimiser settings).

Model	Highest test (%)	Epoch	Mean last 10 (%)	Final train (%)	Params
DeepConvNet, ELU	80.37	131	77.60	86.85	150,977
DeepConvNet, ReLU	**82.04**	99	**79.67**	79.35	150,977
DeepConvNet, LeakyReLU	81.94	115	79.17	79.81	150,977
EEGNet, ELU α=1 (baseline)	81.57	71	79.04	88.89	17,874
EEGNet, ReLU	86.11	92	83.81	95.09	17,874
EEGNet, best (round 2)	88.06	234	86.99	97.04	17,874

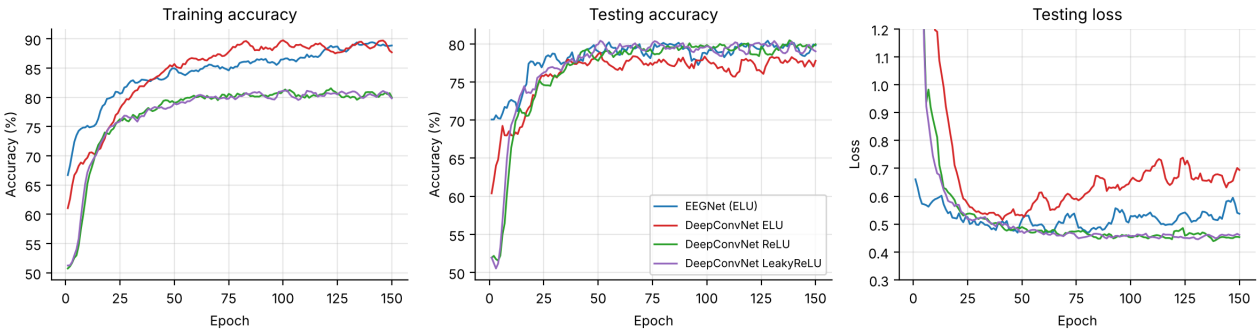


Figure 11: DeepConvNet with three activation functions compared with the EEGNet baseline (5-epoch moving average).

The best DeepConvNet reached 82.04% with ReLU. That is comparable to the EEGNet baseline, but clearly below EEGNet with ReLU (86.11%) and the best EEGNet (88.06%), even though DeepConvNet has 8.4× more parameters. Its training accuracy stayed at only 79–87%. Three dropout layers with $p = 0.5$, weight decay, and a large learning rate together make this deep network hard to fit with ReLU and LeakyReLU. With ELU it fits better (86.85%), but its testing loss rises after about epoch 40, i.e. it starts to overfit (Figure 11). On DeepConvNet the activation ranking is also different: ReLU and LeakyReLU beat ELU ($\alpha = 1$). This agrees with the round-1 EEGNet result that a strongly negative ELU branch is not helpful here. Overall, EEGNet's compact, EEG-specific design reaches higher accuracy with far fewer parameters, which is the main argument of the EEGNet paper [1].

References

[1] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung, and B. J. Lance, "EEGNet: A compact convolutional neural network for EEG-based brain–computer interfaces," *Journal of Neural Engineering*, vol. 15, no. 5, 056013, 2018.

[2] R. T. Schirrmeister *et al.*, "Deep learning with convolutional neural networks for EEG decoding and visualization," *Human Brain Mapping*, vol. 38, no. 11, pp. 5391–5420, 2017.

[3] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)," in *ICLR*, 2016.

[4] BCI Competition III, data set IIIb, http://www.bbci.de/competition/iii/desc_IIIb.pdf.